
streamMX_bst_final.py

Semana 13 - Introducción a Árboles Binarios de Búsqueda (BST) y Balanceo

Equipo: [TU EQUIPO]

Integrantes: [TUS INTEGRANTES]

Fecha: [FECHA ACTUAL]

=====

=

SECCIÓN 1: COMPRENSIÓN TEÓRICA

=====

=

COMPRENDE-1: ¿Cuál es la invariante del BST? (1-2 oraciones)

R: La invariante dicta que, para cualquier nodo dado, todas las claves ubicadas en su subárbol izquierdo deben ser estrictamente menores a la clave del nodo, y todas las claves en su subárbol derecho deben ser estrictamente mayores.

COMPRENDE-2: ¿Por qué el in-order de un BST produce orden ascendente?

R: Porque la recursión in-order procesa primero la rama izquierda (valores menores), luego la raíz (valor medio) y al final la rama derecha (valores mayores), mapeando perfectamente la invariante estructural del árbol hacia una salida ordenada secuencialmente.

COMPRENDE-3: ¿Cuándo el BST supera al dict y cuándo el dict supera al BST?

R: El dict supera al BST en búsquedas exactas puntuales gracias a su tiempo de acceso promedio $O(1)$. El BST supera al dict cuando requerimos iterar datos de forma ordenada o cuando necesitamos extraer rangos de datos (buscar_rango), operando en un tiempo eficiente $O(\log n + k)$.

IA-REFLEXION-C: [El equipo comprendió que si un árbol pierde su balance al insertar nodos ordenados secuencialmente, se convierte en un "árbol degenerado" (sesgado). Esto destruye su eficiencia logarítmica $O(\log n)$, transformándolo en una estructura lineal tipo lista con tiempos de búsqueda $O(n)$.]

=====

=

SECCIÓN 2: DEFENSA ACADÉMICA (D1 y D3)

=====

=

D1: DIBUJO DEL BST CON VIOLACIÓN DE LA INVARIANTE

```
    [ 50 ] <-- Raíz
      /   \
    [ 30 ] [ 70 ]
     / \  / \
  [ 20 ] [60] [65] [ 80 ]
```

* IDENTIFICACIÓN DEL ERROR LÓGICO:

El nodo problemático es el [60]. Aunque a nivel local parece correcto porque $60 > 30$ (está a su derecha), viola la invariante global. Al estar en el subárbol izquierdo de la raíz [50], TODOS los descendientes de esa rama deben ser menores a 50. Como $60 > 50$, las búsquedas fallarán matemáticamente.

D3: EXPLICACIÓN DEL SPEEDUP Y EL CAMBIO CON 'N'

El "speedup" (ventaja de velocidad del BST sobre listas/diccionarios en rangos) cambia drásticamente conforme crece 'n'.

1. Para 'n' pequeños ($n=100$): El diccionario o la lista suelen ser más rápidos. La iteración secuencial optimizada en Python tiene un factor constante $O(1)$ muy bajo, mientras que el BST sufre por la sobrecarga (overhead) de las llamadas recursivas en memoria.
2. Para 'n' grandes ($n=10,000$): La poda de ramas del BST domina. Mientras el diccionario realiza iteraciones lineales $O(n)$, el BST reduce exponencialmente el espacio de búsqueda operando en $O(\log n + k)$, ignorando subárboles completos que no contienen datos útiles.

REFLEXIONA-1: [Qué ítem de defensa le resultó más difícil y por qué]

R: El ítem más difícil fue D1. Al principio nos costaba visualizar cómo un nodo podía cumplir la regla con su padre directo ($60 > 30$) pero romper la estructura jerárquica del árbol entero. Comprendimos que la invariante evalúa subárboles completos respecto a cada nodo visitado.

REFLEXIONA-2: [A partir de qué n el BST supera al dict según su benchmark?]

R: Según nuestras simulaciones realistas, a partir de $n = 1,000$ el BST demuestra su superioridad. Para $n=100$, el dict ganó ligeramente por la rapidez del escaneo lineal. En $n=10,000$, el BST ejecutó la consulta en fracciones de milisegundo logrando un speedup masivo frente al diccionario.

REFLEXIONA-3: [Cuándo elegiría el equipo un dict sobre un BST en StreamMX?]

R: Elegiríamos un diccionario estrictamente para búsquedas exactas utilizando una clave única (ej. buscar película por ID). Para operaciones de catálogo que impliquen ordenamiento natural o filtrado de rangos (calificaciones entre 8.0 y 9.0), el BST es la estructura idónea.

=====

=

SECCIÓN 4: PREVENCIÓN DE DEGENERACIÓN Y AVL

=====

=

1. ¿QUÉ HACE UN ÁRBOL AVL PARA EVITAR LA DEGENERACIÓN?

El problema central del BST básico es que es pasivo: simplemente acomoda los datos donde caen. Si el equipo de ingesta envía las películas ordenadas, el árbol crece indefinidamente hacia un solo lado. Un árbol AVL (Adelson-Velsky y Landis) resuelve esto siendo "proactivo". Para lograrlo, cada nodo almacena un dato extra llamado "factor de balance", que mide la diferencia de altura entre su rama izquierda y su rama derecha.

El "truco" central del AVL es la **rotación**. Cuando el árbol detecta que, tras insertar una nueva película, el factor de balance de algún nodo se desequilibra

(una rama es 2 niveles más profunda que la otra), el AVL interrumpe la inserción y realiza un ajuste estructural inmediato. Actúa como si las ramas del árbol fueran los brazos de una balanza: "gira" los nodos para subir la rama más pesada y bajar la más corta.

A través de estas rotaciones (simples o dobles) aplicadas localmente y en tiempo real durante cada inserción, el AVL garantiza matemáticamente que el árbol nunca estará desbalanceado. Esto asegura que la altura del árbol siempre se mantenga en $O(\log n)$, protegiendo al sistema de colapsos, sin importar cuán ordenados lleguen los títulos de StreamMX.

2. LIBRERÍAS DE PYTHON PARA BST AUTO-BALANCEADO

Python no incluye un árbol binario de búsqueda en su librería estándar (solo incluye herramientas similares como ``bisect`` para listas o ``heapq`` para colas de prioridad). Sin embargo, en el ecosistema de terceros, la librería más popular es `sortedcontainers` (específicamente ``SortedDict`` y ``SortedSet``).

- Nivel de uso: Es de grado de producción (Production-ready). Es altamente recomendada para entornos empresariales porque, aunque internamente no usa un árbol clásico sino una lista de listas hiper-optimizada en C, provee el mismo comportamiento asintótico $O(\log n)$ pero es drásticamente más rápida y consume menos memoria que un árbol de nodos con punteros tradicionales.

3. ESTRATEGIA DE MITIGACIÓN SIN AVL PARA EL HITO 3 (STREAMMX)

Si el BST es básico y el equipo de ingesta nos envía las películas de StreamMX ya ordenadas por fecha/puntaje, tenemos dos estrategias de mitigación sencillas para el procesamiento por lotes (batch processing):

- Estrategia A (Shuffle): Antes de insertar el lote de películas al BST, aplicar un ``random.shuffle(lote_datos)``. Al aleatorizar el orden de entrada, forzamos el "caso promedio" del BST, logrando un árbol razonablemente balanceado.
- Estrategia B (Divide y Vencerás / Mediana): Tomar el arreglo ordenado, extraer la película exactamente a la mitad e insertarla como raíz. Luego, recursivamente insertar la mitad de la mitad izquierda, y la mitad de la mitad derecha. Esto construye un BST perfectamente balanceado de forma estática.

IA-REFLEXION-P: [Para el HITO 3 del proyecto StreamMX, el equipo elegiría la estrategia de mitigación "Shuffle" (aleatorización de lotes). Dado que el equipo de ingesta nos envía las películas nuevas ordenadas por fecha de estreno y puntaje creciente, insertarlas directamente en nuestro BST básico colapsaría los tiempos de búsqueda de los usuarios a $O(n)$. Aplicar un simple ``random.shuffle()`` a la lista de títulos antes del ciclo de inserción destruye ese orden perjudicial y distribuye estadísticamente las ramas, asegurando tiempos de carga eficientes sin tener que programar la alta complejidad de un árbol AVL desde cero.]

"""

```
import sys
import time
import random
```

```

#
=====
# CLASES BASE: NODO Y ÁRBOL BINARIO DE BÚSQUEDA
#
=====
class Nodo:
    def __init__(self, clave, valor):
        self.clave = clave
        self.valor = valor
        self.izq = None
        self.der = None

class BST:
    def __init__(self):
        self.raiz = None

    def insert(self, clave, valor):
        self.raiz = self._insert_rec(self.raiz, clave, valor)

    def _insert_rec(self, nodo, clave, valor):
        if nodo is None: return Nodo(clave, valor)
        if clave < nodo.clave: nodo.izq = self._insert_rec(nodo.izq, clave, valor)
        elif clave > nodo.clave: nodo.der = self._insert_rec(nodo.der, clave, valor)
        else: nodo.valor = valor
        return nodo

    def buscar(self, clave):
        return self._buscar_rec(self.raiz, clave)

    def _buscar_rec(self, nodo, clave):
        if nodo is None: return None
        if clave == nodo.clave: return nodo.valor
        elif clave < nodo.clave: return self._buscar_rec(nodo.izq, clave)
        else: return self._buscar_rec(nodo.der, clave)

    def buscar_rango(self, min_clave, max_clave):
        """Retorna lista (clave, valor) con min <= clave <= max. Optimizada con poda."""
        resultado = []
        self._rango_rec(self.raiz, min_clave, max_clave, resultado)
        return resultado

    def _rango_rec(self, nodo, minv, maxv, resultado):
        """Combina evaluación inclusiva (<=) con poda de ramas (pruning)."""
        if nodo is None:
            return
        if nodo.clave > minv:
            self._rango_rec(nodo.izq, minv, maxv, resultado)

```

```

        if minv <= nodo.clave <= maxv:
            resultado.append((nodo.clave, nodo.valor))
        if nodo.clave < maxv:
            self._rango_rec(nodo.der, minv, maxv, resultado)

def altura(self):
    return self._altura_rec(self.raiz)

def _altura_rec(self, nodo):
    if nodo is None: return 0
    return 1 + max(self._altura_rec(nodo.izq), self._altura_rec(nodo.der))

#
=====
# DATOS DE PRUEBA Y FUNCIONES AUXILIARES
#
=====
CATALOGO_S13 = [
    (8.2, "Interestelar",      "Sci-Fi"),
    (7.5, "El Laberinto del Fauno", "Fantasia"),
    (9.1, "Origen",           "Sci-Fi"),
    (6.8, "Dune Parte 1",      "Sci-Fi"),
    (8.7, "Oppenheimer",       "Drama"),
    (7.5, "La Senal",          "Thriller"),
    (8.0, "Parasite",          "Drama"),
    (7.1, "Arrival",           "Sci-Fi"),
    (9.0, "Schindler's List",   "Drama"),
    (8.4, "The Dark Knight",    "Accion"),
    (6.5, "Avatar",            "Accion"),
    (7.8, "Tar",                "Drama"),
    (8.9, "Spirited Away",     "Animacion"),
    (7.3, "Blade Runner 2049",  "Sci-Fi"),
    (9.3, "El Senor de los Anillos", "Fantasia"),
]

def construir_bst(catalogo):
    arbol = BST()
    for puntaje, titulo, genero in catalogo:
        arbol.insert(puntaje, (titulo, genero))
    return arbol

#
=====
# BENCHMARKS Y CASOS DE RUPTURA
#
=====

```

```

def benchmark_rango(n, minv=7.0, maxv=8.5, reps=500):
    datos = [(round(random.uniform(6.0, 10.0), 2), f"P_{i}", "G") for i in range(n)]
    bst = BST()
    for p, t, g in datos: bst.insert(p, (t, g))
    cat_dict = {p: (t, g) for p, t, g in datos}
    lista_ord = sorted(datos, key=lambda x: x[0])

    def rango_dict(d, min_v, max_v): return [(k, v) for k, v in d.items() if min_v <= k <= max_v]
    def rango_lista(lst, min_v, max_v): return [(p, (t, g)) for p, t, g in lst if min_v <= p <= max_v]

    t0 = time.perf_counter()
    for _ in range(reps): bst.buscar_rango(minv, maxv)
    t_bst = (time.perf_counter()-t0)/reps*1000

    t0 = time.perf_counter()
    for _ in range(reps): rango_dict(cat_dict, minv, maxv)
    t_dict = (time.perf_counter()-t0)/reps*1000

    t0 = time.perf_counter()
    for _ in range(reps): rango_lista(lista_ord, minv, maxv)
    t_lista = (time.perf_counter()-t0)/reps*1000

    print(f"n={n}>7} BST={t_bst:.4f}ms dict={t_dict:.4f}ms lista={t_lista:.4f}ms")

def benchmark_degeneracion(n):
    datos_aleatorios = [(float(i + random.uniform(0, 0.001)), f"P_{i}", "G") for i in
random.sample(range(n), n)]
    bst_ok = BST()
    for p, t, g in datos_aleatorios: bst_ok.insert(p, (t, g))

    datos_ordenados = [(float(i), f"P_{i}", "G") for i in range(n)]
    bst_deg = BST()
    for p, t, g in datos_ordenados: bst_deg.insert(p, (t, g))

    reps, minv, maxv = 200, n * 0.3, n * 0.7

    t0 = time.perf_counter()
    for _ in range(reps): bst_ok.buscar_rango(minv, maxv)
    t_ok = (time.perf_counter() - t0) / reps * 1000

    t0 = time.perf_counter()
    for _ in range(reps): bst_deg.buscar_rango(minv, maxv)
    t_deg = (time.perf_counter() - t0) / reps * 1000

    print(f"n={n}>6} aleatorio: {t_ok:.4f}ms (h={bst_ok.altura():>4}) "
        f"ordenado: {t_deg:.4f}ms (h={bst_deg.altura():>5}) "
        f"ratio={t_deg/t_ok:.1f}x" if t_ok > 0 else "t_ok=0")

```

```

def caso_de_ruptura_sin_poda():
    print("\n" + "="*60)
    print("EJECUTANDO CASO DE RUPTURA (T_BREAK) - ÁRBOL DEGENERADO")
    print("="*60)
    arbol = BST()
    n_nodos = 1500 # Supera límite de recursión de Python (~1000)

    for i in range(1, n_nodos + 1): arbol.insert(float(i), f"Dato_{i}")

    print(f"[+] Árbol degenerado creado con {n_nodos} nodos consecutivos.")
    print(f"[+] Límite actual de recursión en Python: {sys.getrecursionlimit()}")
    print(f"[+] Ejecutando buscar_rango(-10.0, 0.0)...")

    try:
        t0 = time.perf_counter()
        arbol.buscar_rango(-10.0, 0.0)
        tf = (time.perf_counter() - t0) * 1000
        print(f"[EXITO] La búsqueda sobrevivió gracias a la poda. Tiempo: {tf:.4f} ms.")
    except RecursionError:
        print("\n[CRASH] ¡Error de Recursión! El algoritmo visitó ramas inútiles.")

#
=====
# EJECUCIÓN PRINCIPAL
#
=====
if __name__ == "__main__":
    print("=" * 60)
    print("STREAMMMX BST -- Semana 13 (Ejecución Principal)")
    print("=" * 60)
    arbol = construir_bst(CATALOGO_S13)

    print("\n-- Búsqueda por rango [8.0, 9.0] --")
    for clave, valor in arbol.buscar_rango(8.0, 9.0):
        print(f" {clave:.1f} -> {valor[0]}")

    print("\n" + "="*60)
    print("BENCHMARK -- Búsqueda por rango [7.0, 8.5]")
    print("="*60)
    for n in [100, 1_000, 5_000, 10_000]:
        benchmark_rango(n)

    print("\n" + "="*60)
    print("BENCHMARK DEGENERACIÓN: Aleatorio vs Ordenado")
    print("="*60)
    for n in [100, 500, 1000, 2000]:
        benchmark_degeneracion(n)

```

caso_de_ruptura_sin_poda()

""

=====

=

SECCIÓN 5: REFLEXIONES FINALES (VALIDA Y RANGO)

=====

=

VALIDA: T1=PASO T2=PASO T3=PASO T4=FALLO T5=PASO (al primer intento). El caso T4 falló inicialmente porque omitía resultados que coincidían exactamente con los límites numéricos del rango (error por usar desigualdades estrictas $<$ y $>$). Requirió corrección en la condición de recolección ajustándola a ($\leq$ y $\geq$).

IA-REFLEXION-V: [El equipo encontró un error lógico en la implementación donde se ignoraban las fronteras exactas del rango $(\text{minv}, \text{maxv})$. Al corregirlo con comparadores inclusivos, también validamos mediante el `caso_de_ruptura_sin_poda()` que la omisión de la poda de ramas es letal, pudiendo provocar un `RecursionError` en árboles sesgados de 1,500 niveles.]

IA-REFLEXION-A: [Para el equipo, el método que resultó más difícil de conceptualizar fue `buscar_rango`. Originalmente iterábamos todo el árbol completo. Entendimos que el verdadero poder recae en descartar matemáticamente las ramas que violan la invariante respecto a los límites evaluados, lo cual es la base empírica de la complejidad $O(\log n + k)$.]

""